

Exercise 4.1: Traditional Testing vs. ML Model Testing Name at least four differences between testing traditional software and testing a machine learning model.

- Traditional software is easier to debug because the logic is explicitly written by programmers, whereas ML models learn hidden patterns from data, making debugging more difficult.
- In traditional software, outputs are usually deterministic and predictable for a given input. In ML systems, outputs are probabilistic and confidence-based.
- Traditional software failures are often fixed by correcting code bugs, while ML systems may require improvements in data quality, class balance, training strategy, or evaluation metrics such as recall and precision.
- Traditional software logic remains mostly stable across environments, whereas ML model performance can change significantly depending on environmental conditions and training data distribution.
- Traditional software is usually tested using individual test cases, while ML models are evaluated using dataset-level metrics such as accuracy, precision, recall, and F1-score.

Exercise 4.2: Test Oracles In traditional software testing, a test oracle determines whether a test has passed or failed.

1. What plays the role of the test oracle in ML model testing?

In machine learning model testing, the labeled output data acts as the test oracle. The model predictions are compared with the actual labels in the dataset to determine whether the prediction is correct or wrong.

2. Why is defining a good test oracle particularly difficult for perception tasks?

In perception tasks, it is difficult to define a perfect test oracle because real-world input data such as camera images can be blurry, noisy, partially hidden, or unclear. Sometimes even humans may disagree on the correct label. Therefore, it is not always possible to say that the prediction is 100% objectively correct.

Exercise 4.3: From Risk to Metric

Consider binary pedestrian detection in autonomous driving.

1) Write the empirical risk minimization (ERM) objective using cross-entropy loss.

$$\hat{R}(\theta) = \frac{1}{N} \sum_{n=1}^N \ell_{CE}(f_{\theta}(x_n), y_n)$$

where:

- x_n is the input image,
- y_n is the actual label,
- $f_{\theta}(x_n)$ is the model prediction,
- ℓ_{CE} is the cross-entropy loss,
- and N is the total number of training samples.

This objective tries to reduce the average prediction error over the training dataset.

2. Why do we optimize cross-entropy loss instead of recall directly?

Recall cannot be optimized directly because it is not smooth and differentiable. Neural networks learn using gradients, so we need a mathematical function that changes smoothly during training. Cross-entropy loss provides this smooth optimization process and helps the model learn effectively.

3. If a model has low training loss (high accuracy) but low recall on the pedestrian class, what might explain this? (Hint: think about class imbalance and what accuracy actually measures.)

This can happen because the dataset may contain many more “no pedestrian” images than pedestrian images. The model may learn to predict “no pedestrian” most of the time and still achieve high accuracy. However, it may fail to detect actual pedestrians, leading to low recall and dangerous false negatives.

4. How would you detect this problem in practice? What metrics would you compute?

I would not rely only on accuracy. I would compute:

- precision,
- recall,
- F1-score,
- and confusion matrix.

These metrics help identify whether the model is missing pedestrians frequently, even when overall accuracy appears high.

Exercise 4.4: Distribution Shift Types Consider the following three scenarios involving the CARLA autonomous driving model (trained on sunny daytime data):

1. The model is deployed during winter, when roads are often wet and the sun is at a low angle, creating glare in camera images.
2. A new city zone is added to the test route where 60 % of road users are cyclists - a class that was rare (< 5 %) in the original training data.
3. The city replaces all old traffic light housings with a new, slimmer design that the model has never seen.

For each scenario:

- a) Identify which type of distribution shift applies (covariate shift, label shift, or concept shift). Justify your answer.
- b) Explain what effect you would expect on model performance.
- c) Suggest one mitigation strategy

Scenario 1 — Winter roads, glare, wet roads

a) Type of shift:

This is a covariate shift because the input environment changes (wet roads, glare, winter lighting), but the objects like pedestrians and vehicles still remain the same.

b) Effect on model:

The model may give lower accuracy and recall because it was mainly trained on sunny daytime images and may struggle in winter conditions.

c) Mitigation:

Train the model with more winter, wet-road, and glare images or use data augmentation to simulate these conditions.

Scenario 2 — Cyclists increase from 5% to 60%

a) Type of shift:

This is a label shift because the proportion of classes changes. Cyclists were rare during training but became very common during deployment.

b) Effect on model:

The model may not perform well in cyclist-heavy areas and may misclassify cyclists because it has seen fewer examples during training.

c) Mitigation:

Collect more cyclist data and rebalance the dataset before retraining the model.

Scenario 3 — New slim traffic light design

a) Type of shift:

This is a concept shift because the learned appearance and pattern of traffic lights changes compared to the training data.

b) Effect on model:

The traffic light detection model may fail to recognize the new design correctly, reducing precision and recall.

c) Mitigation:

Update the training dataset with the new traffic light designs and retrain or fine-tune the model.

Exercise 4.5: ODD Coverage with k-Projection Coverage You defined an ODD in Exercise Sheet 2. Now assess how well your test set covers this ODD using k-projection coverage.

1. Describe in your own words what k-projection coverage measures and why it is a useful metric for ODD coverage.

- k-projection coverage measures how many different ODD condition combinations are covered by the test dataset. Instead of checking all possible combinations together, it checks smaller combinations such as single conditions ($k=1$), pairs of conditions ($k=2$), or three-condition combinations ($k=3$).
- It is useful because autonomous driving systems may fail under specific combinations like rain + night + heavy traffic. k-projection coverage helps identify whether the dataset contains enough variety of conditions to properly test the model inside the defined ODD.

2) Compute the k-projection coverage of your test set for $k \in \{1, 2, 3\}$.

The k-projection coverage was computed for $k = 1, 2$, and 3 using the defined ODD conditions and test scenarios.

- For $k = 1$, the coverage was 1.0 (100%), which means all individual conditions such as weather, lighting, and road type were covered by the test set.
- For $k = 2$, the coverage decreased to 0.67, showing that some pairs of conditions were missing from the dataset.
- For $k = 3$, the coverage further decreased to 0.375, meaning only a few complex three-condition combinations were present in the test set.

This shows that the dataset covers simple individual conditions well, but it does not fully cover more complex real-world combinations. As k increases, coverage decreases because more combinations of ODD conditions need to be represented. This indicates that the test set may still miss some important deployment scenarios.

3. How does coverage change with increasing k ? What does this tell you about the adequacy of your test set?

- The coverage decreases as the value of k increases. For $k = 1$ the coverage was 100%, for $k = 2$ it reduced to 66.7%, and for $k = 3$ it further reduced to 37.5%.
- This happens because higher k values require more complex combinations of ODD conditions to be present in the dataset. While the test set covers individual conditions well, it does not contain enough combinations of multiple conditions together.
- This indicates that the test set is only partially adequate. It is sufficient for basic condition testing, but it may not fully represent complex real-world driving situations inside the ODD.

Exercise 4.6: Designing a Test Suite from Safety Constraints Refer back to Exercise 2.7 in Sheet 2, where you derived safety constraints from Unsafe Control Actions (UCAs). For each model-level constraint, design one test case using the following structure:

Constraint ID	Constraint	Test input description	Expected output	Pass criterion
SC-1	The system must apply braking when a pedestrian is detected in front	A pedestrian suddenly crosses the road in front of the vehicle during daytime driving	The vehicle should detect the pedestrian and apply braking	Vehicle successfully brakes without missing the pedestrian
SC-2	The system should not brake unless a valid obstacle is detected	The vehicle is driving on a clear road with no pedestrians or vehicles nearby	The system should continue driving normally without unnecessary braking	No false braking should occur
SC-3	The system must respond within a safe time after detecting an obstacle	A pedestrian appears close to the vehicle while driving at moderate speed	The system should detect the obstacle and react quickly	Braking response happens within safe reaction time
SC-4	The system must continue braking until the path is clear	A moving pedestrian remains in front of the vehicle for several seconds	The vehicle should continue braking until the pedestrian leaves the path	Vehicle should not move while pedestrian is still present

Exercise 4.7: Per-Class Evaluation Evaluate each of your three models on the provided test set.

1. Compute and report precision, recall, and F1-score for each model.
2. Plot a confusion matrix for each model.

3. Which model has the lowest recall? Is this the model you expected to be hardest based on your hazard analysis?

- The pedestrian detection model had the lowest recall.
- Yes, this was the model expected to be the hardest based on the hazard analysis because missing a pedestrian can directly lead to human injury or death. Pedestrians are also more difficult to detect since they may be small, partially hidden, or moving unpredictably in different environments.

4. Based on a safety argument: what minimum recall would you require for the pedestrian model before considering deployment? Justify your threshold.

Before considering deployment, I would require at least 95% recall for the pedestrian detection model. In autonomous driving, false negatives are very dangerous because the system may fail to detect a pedestrian and continue driving. A high recall is important to reduce the chance of missing pedestrians, even if it slightly increases false positives.